

KICC 3D BILLBOARD PLATFORM

Complete Pipeline Architecture & Software Requirements

7-Use-Case Implementation Plan

Images In | AI 3D Creation Prompt | Video Out

VR Booths | Curved Screen 3D | Airport Immersive Displays

Phone 3D Viewing | Mobile VR-like Explorer | Live 3D Streaming

AI Pipeline: Kimi K3 | Nano Banana 2 | Higgsfield

KICC Digital Economy Platform

Kenya National Exhibition Centre

Nairobi, Kenya

Document generated: July 2026

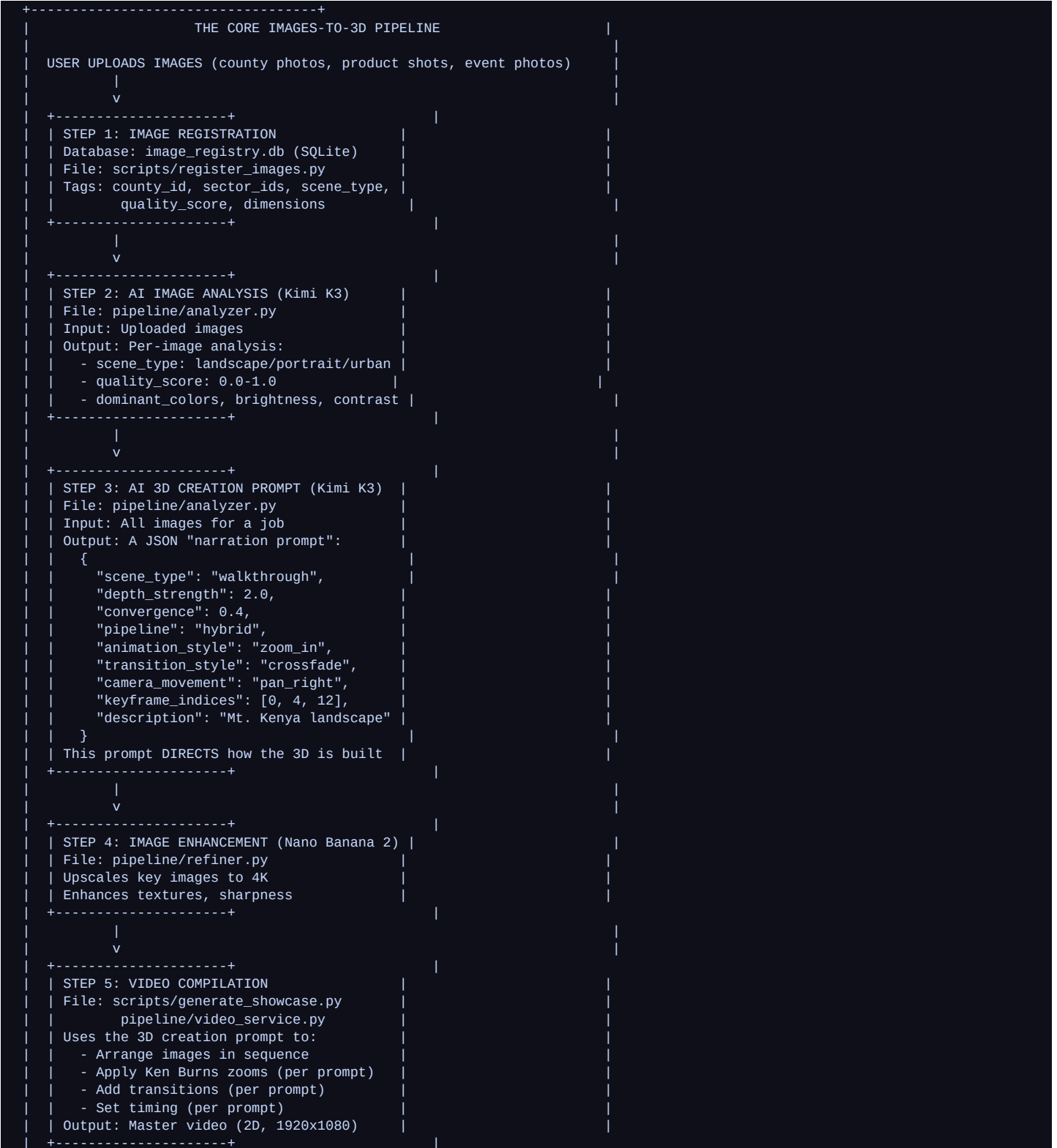
. TABLE OF CONTENTS

- 1. Core Architecture - Images In, 3D Creation Prompt, Video Out**
 - 2. Use Case 1: VR Booth - Site Experiences (Mt. Kenya)**
 - 3. Use Case 2: Curved Screen 3D Realism - Mechanical Assembly**
 - 4. Use Case 3: Airport Curved Screen - Immersive History of Kenya**
 - 5. Use Case 4: AI Software Pipeline - Kimi K3, Nano Banana 2, Higgsfield**
 - 6. Use Case 5: Phone Viewing - All 3D Videos on Mobile**
 - 7. Use Case 6: Phone 3D Explorer - VR-like Interaction on Phone**
 - 8. Use Case 7: Live 3D Exhibition Streaming**
 - 9. Pipelines to Build - Detailed Implementation Plans (5 Pipelines)**
 - 10. Complete Software Stack - Paid vs Free, All Use Cases**
 - 11. Cost Summary - Monthly Run Rate & One-Time Purchases**
 - 12. Priority Build Order - Easiest to Most Complex**
- Appendix A: Existing Pipeline File Reference**
- Appendix B: Integration Architecture - How Everything Connects**

1. CORE ARCHITECTURE

The KICC 3D Billboard Platform is built on a single fundamental principle: **USERS UPLOAD IMAGES**, and the **PIPELINE COMPILES THEM INTO VIDEOS** guided by an **AI-generated 3D CREATION PROMPT**. The prompt tells the pipeline how to build the 3D experience - what depth to use, what animation style, what perspective. Real human voiceover artists record narration separately where needed. The AI does not narrate; it **DIRECTS** the 3D rendering.

1.1 The Core Flow - Images to 3D Video



```
|
|      v
| +-----+
| | STEP 6: 3D CONVERSION
| | File: pipeline/stereo_worker.py
| |     pipeline/splat_worker.py
| | Based on the 3D creation prompt:
| |   - If depth_strength > 0: SBS 3D video
| |   - If pipeline = "splat": 3DGS scene
| |   - If pipeline = "hybrid": both
| +-----+
|      v
| +-----+
| | STEP 7: DELIVERY
| | Upload to S3/CDN
| | Create HLS adaptive stream for phones
| | Make available for VR headset download
| | Send to screen CMS for billboard playback
| +-----+
|
| [SEPARATE] HUMAN VOICEOVER ARTIST records narration
| Audio is mixed into the final video as a separate step
| +-----+
```

1.2 What Already Exists vs What Needs Building

COMPONENT	STATUS	FILE(S)
Image upload & registration	EXISTS	register_images.py
Image analysis (Pillow)	EXISTS	image_analyzer.py
Image analysis (Kimi K3)	EXISTS	analyzer.py
3D creation prompt (Kimi K3)	EXISTS	analyzer.py
Image enhancement (Imagen)	EXISTS	refiner.py
Video compilation	EXISTS	generate_showcase.py
Dynamic video service	EXISTS	video_service.py
SBS 3D video generation	EXISTS	stereo_worker.py
Gaussian Splatting	SCAFFOLDED	splat_worker.py
3DGS training (full)	NEEDS BUILD	splat_worker.py
Splat compression for web	NEEDS BUILD	gsplat integration
HLS adaptive streaming	NEEDS BUILD	hls_packager.py
PWA offline support	NEEDS BUILD	manifest.json, sw.js
Gyroscope/joystick controls	NEEDS BUILD	mobile_3d_viewer.js
TouchDesigner pixel mapping	NEEDS BUILD	.toe files
Live 3D streaming	NEEDS BUILD	8 new files
AI content router	NEEDS BUILD	ai_content_router.py
Higgsfield AI adapter	NEEDS BUILD	higgsfield_adapter.py
Screen geometry database	NEEDS BUILD	migration + model
Filament resources	NEEDS BUILD	7+ resources

1.3 The 3D Creation Prompt - The Brains of the System

This is the most important concept. When a user uploads images, Kimi K3 analyzes them and generates a JSON prompt that tells the entire pipeline how to create the 3D experience. This prompt is NOT a voiceover narration. It is a TECHNICAL DIRECTIVE for the 3D rendering engine. A real human voiceover artist records spoken narration separately. The AI directs the 3D; the human does the voice.

```
EXAMPLE 3D CREATION PROMPT (generated by Kimi K3):

{
  "job_id": "job_20260724_001",
  "content_title": "Mt. Kenya Sunrise Experience",

  "scene_type": "walkthrough",
  "description": "A series of landscape photos showing Mt. Kenya at sunrise, with snow-capped peak, alpine meadows, and forest foothills",

  "depth_strength": 2.0,
  "convergence": 0.4,
  "pipeline": "hybrid",

  "animation_style": "ken_burns",
  "camera_movement": "zoom_in_to_peak",
  "transition_style": "crossfade",
  "clip_duration_per_image": 4.0,
  "total_duration_seconds": 120,
```

```
"screen_placement": {  
  "screen_type": "curved_L",  
  "sweet_spot_angle": 15,  
  "viewing_distance_m": 5.0  
},  
  
"billboard_3d": {  
  "inverted_box": true,  
  "virtual_room_depth": 3.0,  
  "border_break_objects": ["mountain_peak", "sun"],  
  "forced_perspective": true  
},  
  
"keyframe_indices": [0, 5, 15, 30],  
"enhancement_level": "high",  
  
"target_formats": ["sbs_3d", "anaglyph", "webgl_splat", "flat_2d"]  
}
```

This prompt is consumed by:

- generate_showcase.py: how to sequence and animate the images
- stereo_worker.py: how to create SBS 3D depth
- splat_worker.py: whether to build interactive 3D scene
- TouchDesigner: how to map to curved screen
- video_service.py: which images to select, what duration

2. USE CASE 1: VR BOOTH - SITE EXPERIENCES

Goal: A person enters a physical VR booth at KICC, puts on a VR headset, and experiences a Kenyan site like Mt. Kenya. The experience is built from USER-UPLOADED IMAGES of that site, compiled into an immersive 3D video guided by the AI 3D creation prompt.

2.1 Pipeline: Images to VR Experience

```
USER UPLOADS: 15-30 photos of Mt. Kenya (different angles, times of day)
|
v
[1] IMAGE REGISTRATION (scripts/register_images.py)
    Tags: county_id=meru, scene_type=mountain, quality_score=0.85
    |
    v
[2] AI ANALYSIS (pipeline/analyzer.py - Kimi K3)
    Khimi identifies: "landscape, mountain, snow, forest, sunrise"
    Generates 3D creation prompt:
    - depth_strength: 2.0 (strong pop-out)
    - pipeline: hybrid (SBS + splat)
    - animation: zoom_in_to_peak
    - inverted_box: true
    |
    v
[3] IMAGE ENHANCEMENT (pipeline/refiner.py - Nano Banana 2)
    Best 5 keyframes upscaled to 4K
    |
    v
[4] VIDEO COMPILATION (scripts/generate_showcase.py)
    Compiles images into 120s master video:
    - Title card: "Mt. Kenya - The Sacred Mountain"
    - Ken Burns zoom into peak
    - Crossfade transitions between angles
    - Labels per image (sector, elevation)
    Output: mt_kenya_master.mp4 (1920x1080, 30fps)
    |
    +----[5a] SBS 3D CONVERSION (pipeline/stereo_worker.py)
    |         Converts 2D video to SBS 3D
    |         Uses depth_strength=2.0 from AI prompt
    |         Output: mt_kenya_SBS.mp4 (3840x1080)
    |         Viewable in: VR headset (full 3D)
    |
    +----[5b] 3D SPLAT RECONSTRUCTION (pipeline/splat_worker.py)
    |         Builds interactive 3D scene from images
    |         Output: mt_kenya.splat
    |         Viewable in: WebGL browser (drag/gyro to look)
    |
    v
[6] VR DELIVERY
    - SBS video plays in VR headset (WebXR or Unity)
    - User sees true 3D depth between foreground and background
    - Look around by turning head (360° for splat scenes)
    - [SEPARATE] Human voiceover artist records narration
    - Narration audio mixed into final VR experience

[7] PHONE DELIVERY (parallel)
    - HLS adaptive stream from master video
    - Anaglyph 3D mode for red/blue glasses
    - Interactive splat mode: tilt phone to look around
```

2.2 Software Requirements

Software	Cost	Type	Purpose
Kimi K3 (OpenRouter)	\$50-200/mo	AI API	Image analysis + 3D creation prompt
Nano Banana 2 (Imagen)	\$20/mo	AI API	Image enhancement to 4K
Depth Anything V2	FREE	Open-source	Depth map for SBS conversion
3D Gaussian Splatting	FREE	Open-source	Interactive 3D scene from images
COLMAP	FREE	Open-source	Camera pose for multi-image splatting
FFmpeg	FREE	Open-source	Video compilation, Ken Burns, transitions
Three.js + WebXR	FREE	CDN	Browser VR/3D rendering
Unity Pro (optional)	\$2,040/yr	Paid	Native VR app (higher quality)
Meta Quest 3	\$500-700	Hardware	VR headset per booth

Vast.ai A100 GPU	\$500-1,000/mo	Cloud GPU	DAv2, 3DGS training
------------------	----------------	-----------	---------------------

2.3 What Needs To Be Built

1. Connect analyzer.py output (3D creation prompt) directly to generate_showcase.py so prompt dictates animation style, transitions, timing
2. Complete splat_worker.py train_3dgs() - currently placeholder, needs actual 3DGS training code
3. Add gsplat compression for web delivery of splat scenes
4. Extend booth_viewer.html with WebXR VR mode
5. Build VR booth management UI in Filament (select images, trigger pipeline, preview result)

3. USE CASE 2: CURVED SCREEN 3D REALISM - MECHANICAL ASSEMBLY

Goal: On a large curved LED screen, show a car being assembled from individual parts, or a coffee bean becoming a cup of coffee. The content is built from USER-UPLOADED IMAGES of each step, compiled into a video with the 3D billboard effect (inverted box illusion, forced perspective from the sweet spot).

3.1 Pipeline: Images to Anamorphic 3D Billboard

```

USER UPLOADS: Photos of each car part, assembly steps, final product
|
v
[1-3] Same as Core Pipeline: Register -> Analyze -> Enhance
    Khimi generates 3D creation prompt:
    - scene_type: "mechanical_assembly"
    - animation_style: "sequential_build"
    - transition_style: "parts_fly_together"
    - inverted_box: true
    - border_break_objects: ["engine_block", "wheels"]
    |
    v
[4] VIDEO COMPILATION (generate_showcase.py)
    Compiles images into master video:
    - Part 1: Title card "How a Car is Built"
    - Part 2: Each part appears sequentially
    - Part 3: Parts animate to assemble (requires Houdini/UE5)
    - Part 4: Final product reveal
    |
    v
[5] ANAMORPHIC RENDERING - The 3D Billboard Effect
    The video is rendered with forced perspective so it
    appears 3D when viewed from the sweet spot:
    - Camera positioned at pedestrian eye level
    - Virtual room depth (inverted box illusion)
    - Objects near the frame edge appear to leap out
    - Implemented in: UE5 or Three.js shader
    |
    v
[6] PIXEL MAPPING - TouchDesigner ($50/mo)
    Maps the anamorphic video to the physical curved screen:
    - UV unwrapping: 3D screen surface -> 2D canvas
    - Distortion correction: compensates for curvature
    - Seam blending: hides panel junctions
    - Output: pixel-exact signal to LED processor
    |
    v
[7] LED PROCESSOR - Novastar ($500-5,000 HW)
    |
    v
[8] CURVED LED SCREEN - Final 3D billboard display

```

3.2 The Inverted Box Illusion

The key technique behind Shinjuku cat and Times Square 3D billboards. The software renders a "virtual room" inside the screen's frame. By adding realistic lighting, shadows, and interior walls to the render, the flat screen appears to be a hollow box. When an object (like a car part) moves from inside the box toward the screen edge and crosses the border, it appears to LEAP into the viewer's space. The effect works from a single "sweet spot" - the camera's position in the real world. The 3D creation prompt generated by Kimi K3 tells the system exactly how to build this: `depth strength, convergence, border break objects, virtual room depth.`

3.3 Software Requirements

Software	Cost	Type	Purpose
Blender	FREE	Open-source	3D modeling of parts (if needed)
Houdini Indie	\$50/mo	Paid	Procedural assembly animation
Unreal Engine 5	FREE (5% royalty)	Engine	Anamorphic rendering + inverted box
TouchDesigner Commercial	\$50/mo	Paid	UV unwrapping, pixel mapping, distortion
Novastar LED Processor	\$500-5,000	Hardware	LED panel controller
Kimi K3	\$50-200/mo	AI API	3D creation prompt generation
Nano Banana 2	\$20/mo	AI API	Image enhancement

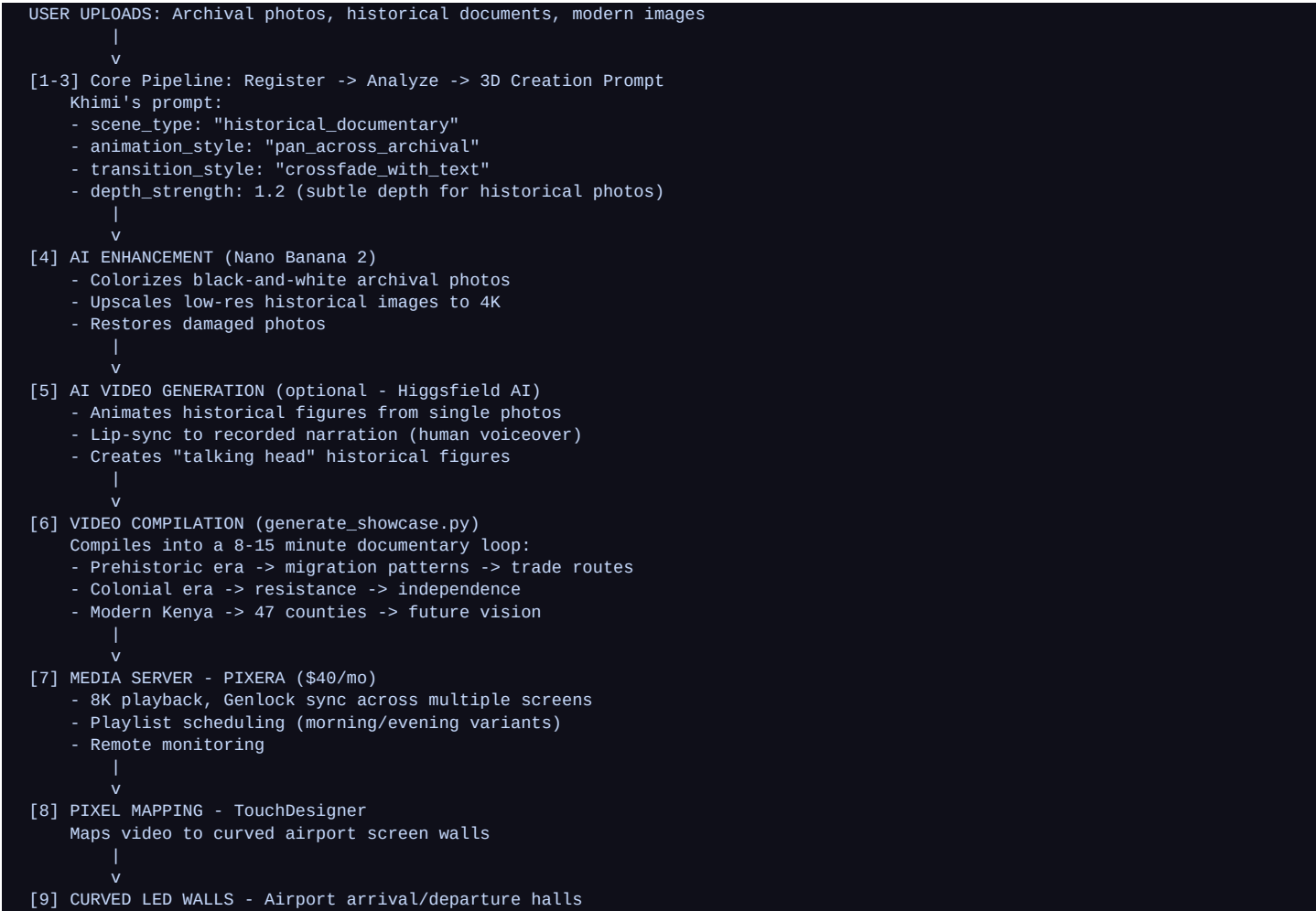
3.4 What Needs To Be Built

1. Screen geometry database - physical dimensions, curvature, sweet spot per screen
2. TouchDesigner pipeline - UV unwrapper, distortion shader, seam blender
3. Inverted box illusion shader - Three.js or UE5 post-process material
4. Connect 3D creation prompt to TouchDesigner - prompt tells TD how to configure the anamorphic rendering

4. USE CASE 3: AIRPORT CURVED SCREEN - IMMERSIVE HISTORY OF KENYA

Goal: Large curved screens at JKIA showing an immersive history of Kenya, built from USER-UPLOADED IMAGES - archival photos, historical documents, modern photos of each era. The AI generates the 3D creation prompt that determines how each image is presented (depth, animation, transitions).

4.1 Pipeline: Images to Airport Immersive Display



4.2 Software Requirements

Software	Cost	Type	Purpose
DaVinci Resolve Studio	\$295 once	Paid	Edit + color grade documentary
After Effects	\$55/mo	Paid	Map animations, text overlays
Google Veo 2.0	\$20/mo	API	AI video generation of historical scenes
Higgsfield AI	\$30-100/mo	API	Animate historical figures from photos
Nano Banana 2	\$20/mo	API	Colorize/restore archival photos
PIXERA	\$40/mo	Paid	Media server, Genlock sync
TouchDesigner	\$50/mo	Paid	Curved screen pixel mapping
Novastar LED Proc	\$500-5,000	Hardware	LED panel control

4.3 What Needs To Be Built

- Higgsfield AI adapter - animate historical figures from archival photos
- Nano Banana 2 archival colorization pipeline - refiner.py extension for B&W photos
- PIXERA integration - API-based playlist management, content upload
- Airport screen CMS dashboard - Filament resource for scheduling, monitoring

5. USE CASE 4: AI SOFTWARE PIPELINE

How the three AI tools - Kimi K3, Nano Banana 2 (Google Imagen), and Higgsfield AI - integrate into the pipeline. The key role of Kimi K3 is to generate the 3D CREATION PROMPT that directs every downstream step.

5.1 Kimi K3 - 3D Creation Prompt Generator (Already Integrated)

```
Model: moonshotai/kimi-k3-mini via OpenRouter
Cost: ~$0.50/analysis -> $50-200/mo
API key: OPENROUTER_API_KEY
Files: pipeline/analyzer.py
       agentic_loop/decider.py
```

WHAT IT DOES - THE CRITICAL ROLE:

- 1. IMAGE ANALYSIS**
 Input: User-uploaded images (county photos, product shots, etc.)
 Output: Per-image analysis: scene_type, quality_score, dominant_colors, brightness, contrast, dimensions
- 2. 3D CREATION PROMPT GENERATION**
 Input: All images for a job + per-image analysis
 Output: JSON prompt that tells the pipeline:
 - scene_type: how to categorize this content
 - depth_strength: how much 3D pop-out (0.5-3.0)
 - convergence: how much depth goes behind screen
 - pipeline: which 3D process to use (sbs/splat/hybrid)
 - animation_style: how to animate the images
 - transition_style: how to transition between images
 - camera_movement: what camera path to use
 - keyframe_indices: which images to enhance
 - description: human-readable summary

This prompt is consumed by EVERY downstream pipeline step:

 - generate_showcase.py: animation, transitions, timing
 - stereo_worker.py: SBS depth parameters
 - splat_worker.py: whether to build 3D scene
 - TouchDesigner: anamorphic billboard configuration
 - video_service.py: image selection, duration calculation
- 3. AUTONOMOUS OPTIMIZATION (agentic_loop/decider.py)**
 Observes pipeline output quality and suggests:
 - "Increase depth_strength for this scene"
 - "Use splat instead of SBS for better results"
 - "Try different transition style"

5.2 Nano Banana 2 - Image Enhancement (Already Integrated)

```
Model: Google Imagen 3.0 (imagen-3.0-generate-001)
Cost: $20/mo (Google AI Pro subscription)
API key: GOOGLE_AI_API_KEY
Files: pipeline/refiner.py
```

WHAT IT DOES:

- Upscales key images to 4K (identified by Kimi's 3D creation prompt)
- Enhances textures, sharpness, detail
- Colorizes black-and-white archival photos
- Restores damaged/old images
- Falls back to OpenCV Lanczos if API unavailable

PIPELINE INTEGRATION:

- Kimi's 3D creation prompt -> keyframe_indices
- > refiner.py enhances those specific images
- > enhanced images used in video compilation (better quality)
- > enhanced images used as starting frames for Veo promos

5.3 Higgsfield AI - Video Generation (NEW, Not Yet Integrated)

```
What it is: AI video generation platform (text-to-video, image-to-video,
            face-swap, lip-sync). Competitor to Runway Gen-3/Pika.

Cost: ~$30-100/mo estimated (pay-per-generation)

NEW FILES TO BUILD:
```

```

USER UPLOADS IMAGES
|
v
+-----+
| KIMI K3 (analyzer.py) |
| 1. Analyzes each image: scene_type, quality, colors |
| 2. Generates 3D CREATION PROMPT: |
|   - "These are mountain landscape photos |
|     Use depth_strength=2.0, pipeline=hybrid, |
|     animation=zoom_in_to_peak, |
|     inverted_box=true, |
|     border_break_objects=[mountain_peak, sun]" |
| 3. Output: JSON prompt consumed by all downstream |
+-----+
|
+-----+ +-----+
|                                     |
v                                     v                                     v
+-----+ +-----+ +-----+
| NANO BANANA 2 | | GENERATE | | SPLAT WORKER | |
| (refiner.py) | | SHOWCASE.py | | (splat_worker) | |
|               | |           | |              | |
| Enhance key   | | Compile images | | Build 3D scene | |
| images to 4K  | | into video    | | from images    | |
| using prompt's | | using prompt's | | using prompt's | |
| keyframe_indices | | animation_style | | pipeline param | |
+-----+ +-----+ +-----+
|               | |           | |              | |
+-----+ +-----+ +-----+
|               | |           | |              | |
v               v               v
+-----+
| DELIVERY |
| - SBS 3D video for VR headset |
| - Anaglyph 3D for phone viewers |
| - Interactive WebGL splat for phone 3D explorer |
| - Flat 2D video for standard screens |
| - [SEPARATE] Human voiceover mixed into final video |
+-----+

```

6. USE CASE 5: PHONE VIEWING - ALL 3D VIDEOS ON MOBILE

Goal: Users can watch any 3D video content - SBS stereoscopic, compiled showcase videos, and interactive 3D scenes - directly on their phone without a VR headset.

6.1 The SBS Problem on Phones

Side-by-side (SBS) 3D video requires a VR viewer (Google Cardboard) to see the stereoscopic effect. Without one, the screen shows two identical images side by side. Three solutions exist for phone delivery:

```
SOLUTION 1: Anaglyph 3D (Red/Blue Glasses)
Process: Convert SBS to red/blue anaglyph via FFmpeg
Cost:    FREE
Quality: Lower (color is compromised)
Hardware: Any phone + cheap red/blue glasses ($2)
FFmpeg:  ffmpeg -i sbs.mp4 -vf "stereo3d=sbsl:aybg" anaglyph.mp4

SOLUTION 2: Interactive WebGL 3D (Best for phone)
Process: Render 3D splat scene in Three.js on phone
Cost:    FREE (Three.js CDN)
Quality: Best (real 3D, user controls camera)
Hardware: Any modern phone with WebGL
Status:  Already works - three HTML files exist

SOLUTION 3: Standard 2D Video (No 3D Effect)
Process: Extract left eye from SBS
Cost:    FREE (FFmpeg crop)
Quality: Good for flat viewing
FFmpeg:  ffmpeg -i sbs.mp4 -vf "crop=iw/2:ih:0:0" left.mp4
```

6.2 Pipeline: Video to Phone Delivery

```
MASTER VIDEO (from generate_showcase.py)
|
+--> HLS ADAPTIVE STREAMING (pipeline/hls_packager.py NEW)
|   FFmpeg packages into 144p, 360p, 720p, 1080p, 4K
|   Output: master.m3u8 playlist
|   Player: hls.js in browser (auto-quality)
|
+--> ANAGLYPH 3D (FFmpeg filter)
|   Output: anaglyph.mp4
|   Player: standard <video> tag
|
+--> LEFT-EYE 2D (FFmpeg crop)
|   Output: flat.mp4
|   Player: standard <video> tag

SPLAT SCENE (from splat_worker.py)
|
+--> gsplat.js COMPRESSION
|   Compress millions of Gaussians to ~300K
|   Output: scene.splat (90% smaller)
|
+--> THREE.JS WEBGL VIEWER
|   Player: mobile_3d_viewer.js with gyroscope/joystick
```

6.3 Software Requirements

Software	Cost	Type	Purpose
FFmpeg	FREE	Open-source	HLS packaging, anaglyph, crop
hls.js	FREE	CDN	HLS playback in browser
Three.js	FREE	CDN	Interactive WebGL 3D viewer
Video.js	FREE	CDN	Mobile video player, touch controls
PWA manifest + sw.js	FREE	Web standard	Offline caching, install on phone
gsplat.js	FREE	CDN	Compressed splat loading on mobile

6.4 What Needs To Be Built

1. Add HLS packaging to video_service.py output stage
2. Add anaglyph + left-eye extraction to stereo_worker.py
3. Add PWA support (manifest.json + service worker)

4. Replace basic <video> tag with hls.js player for adaptive quality

7. USE CASE 6: PHONE 3D EXPLORER - VR-LIKE ON PHONE

Goal: On a phone, the user can explore a 3D scene built from user-uploaded images - drag to look around, tilt phone to pan, pinch to zoom - the same freedom as VR but on a phone screen.

7.1 Pipeline: Images to Mobile 3D Explorer

```

USER UPLOADS: Multiple photos of a location from different angles
|
v
[1-3] Core Pipeline: Register -> Analyze -> 3D Creation Prompt
      Khimi's prompt: pipeline="splat", scene_type="interactive_explore"
      |
      v
[4] 3D GAUSSIAN SPLATTING (pipeline/splat_worker.py)
      Reconstructs a 3D scene from the multiple images
      Output: .ply point cloud -> compressed .splat file
      GPU: Requires A100 on Vast.ai ($500-1000/mo)
      |
      v
[5] SPLAT COMPRESSION (gsplat.js)
      Reduces millions of Gaussians to ~300K for web
      Progressive loading: load visible area first
      |
      v
[6] THREE.JS MOBILE VIEWER (public/3d/)
      Extends existing HTML files with:
      - DeviceOrientation API: tilt phone to look around
      - nipplejs: virtual joystick to move through scene
      - gsplat.js: compressed splat loader
      - Quality auto-detect: adjust for phone RAM
      - Touch-optimized UI: large buttons, bottom sheets
  
```

7.2 Phone Controls

```

+-----+
|          PHONE 3D EXPLORER CONTROLS          |
| | | | | | | | | | | | | | | | | | | | | | | |
| [Tap] Tap to toggle: Orbit mode vs Gyroscope mode |
| [Drag] Drag finger -> rotate view (orbit mode) |
| [Pinch] Pinch -> zoom in/out |
| [Tilt] Tilt phone -> look around (gyro mode) |
| [Joystick] Virtual joystick -> move through scene |
| [Tap object] Tap -> info card + play video |
| | | | | | | | | | | | | | | | | | | | | | | |
| MODE TOGGLE: |
| [2D] Flat video (standard) |
| [3D] Anaglyph (red/blue glasses) |
| [VR] Split screen SBS (Google Cardboard) |
| [Explore] Interactive 3D (gyroscope + joystick) |
| | | | | | | | | | | | | | | | | | | | | | | |
+-----+
  
```

7.3 Software Requirements

Software	Cost	Type	Purpose
3D Gaussian Splatting	FREE	Open-source	3D reconstruction from images
gsplat.js	FREE	CDN	Compressed splat for web
Three.js	FREE	CDN	WebGL rendering + OrbitControls
DeviceOrientation API	FREE	Browser API	Gyroscope (built-in)
nipplejs	FREE	CDN	Virtual joystick
Vast.ai GPU	\$500-1,000/mo	Cloud GPU	Splatting reconstruction

7.4 What Needs To Be Built

1. DeviceOrientation API in kenya_3d_map.html and booth_viewer.html
2. nipplejs virtual joystick for movement through scene
3. gsplat.js integration for compressed splat loading
4. Quality auto-detect (phone RAM -> splat density)
5. Touch-optimized UI (larger buttons, bottom-sheet panels, swipe gestures)

8. USE CASE 7: LIVE 3D EXHIBITION STREAMING

Goal: A live event at KICC is streamed in real-time. Users watch on phone (2D/3D) or VR headset (full SBS). The live stream is captured from dual stereo cameras, not compiled from images - this is the only pipeline that uses live video input instead of user-uploaded images.

8.1 Pipeline: Live Event to 3D Stream

```

LIVE EVENT AT KICC
|
v
[1] CAPTURE - Dual Stereo Camera Rig
    2x genlocked cameras (65mm apart, human eye distance)
    Blackmagic Pocket Cinema Camera or Sony FS5 ($1,500-6,000)
    SDI -> Blackmagic DeckLink Duo 2 ($695) -> PC
    |
    v
[2] COMPOSITE - OBS Studio (FREE)
    Dual feeds arranged side-by-side
    Output: 3840x1080 SBS frame, 60fps
    Audio: venue PA mix (stereo)
    File: livestream/obs_scene_sbs.json (NEW)
    |
    v
[3] STREAM - nginx-rtmp + HLS (FREE)
    RTMP ingest from OBS
    HLS packaging into .ts segments
    LL-HLS for ~3-5s latency
    File: deploy/streaming-server/docker-compose.yml (NEW)
    |
    v
[4] CDN - Cloudflare Stream ($10/mo)
    Pulls HLS stream, transcodes to multiple qualities
    Global CDN distribution
    |
    v
[5] PLAYER - Web VR/3D/2D (public/3d/live_player.html NEW)
    Phone 2D: Left-eye extraction -> standard HLS player
    Phone 3D: SBS HLS -> WebGL anaglyph shader (real-time)
    VR:      SBS HLS -> WebXR split screen
    |
    v
[6] RECORDING - On-Demand Replay
    OBS records locally + Cloudflare DVR
    Available for replay after event ends
  
```

8.2 Software Requirements

Software	Cost	Type	Purpose
OBS Studio	FREE	Open-source	Dual camera capture + SBS composite
nginx-rtmp	FREE	Open-source	RTMP ingest + HLS packaging
Cloudflare Stream	\$10/mo	Paid API	CDN delivery + transcoding
Blackmagic DeckLink Duo 2	\$695	Hardware	Dual SDI capture card
2x Stereo Cameras	\$1,500-6,000	Hardware	Genlocked camera pair
RTX 4090 Streaming PC	\$3,000-5,000	Hardware	Capture + encode workstation
hls.js + Three.js	FREE	CDN	Browser player + anaglyph shader

8.3 What Needs To Be Built

1. OBS scene template for dual camera SBS compositing
2. nginx-rtmp Docker configuration for streaming server
3. Laravel Livestream model + migration + Filament resource + service
4. live_player.html with hls.js + Three.js anaglyph shader + WebXR

9. PIPELINES TO BUILD - DETAILED IMPLEMENTATION PLANS

Five distinct pipelines must be built. Each pipeline is described in full detail: what it does, every file, every step, how it automates, and how it connects to the 3D creation prompt from Kimi K3.

9.1 Pipeline A: 3D Content Pipeline (Core - Images to 3D Video)

PURPOSE: The central pipeline. Takes user-uploaded images, analyzes them with AI, generates a 3D creation prompt, and compiles them into a video with 3D depth effects. Every other pipeline consumes the output of this one.

```
FILES EXISTING:
pipeline/analyzer.py      (124 lines - Kimi K3 analysis + prompt gen)
pipeline/depth_worker.py  (131 lines - DAV2 depth maps)
pipeline/stereo_worker.py (158 lines - SBS 3D generation)
pipeline/splat_worker.py  (146 lines - 3DGS scaffolded)
pipeline/refiner.py       (84 lines - Nano Banana 2 enhancement)
pipeline/resolver.py      (101 lines - DaVinci/FFmpeg grading)
pipeline/orchestrator.py  (201 lines - master controller)
pipeline/video_service.py (288 lines - dynamic video generation)
scripts/generate_showcase.py (252 lines - image->video compilation)

FILES TO CREATE:
pipeline/hls_packager.py  - HLS adaptive streaming packaging
pipeline/delivery.py      - S3 upload + database update + webhook

FILES TO MODIFY:
splat_worker.py - Replace train_3dgs() placeholder with actual training
                - Add compress_splat() for web delivery
stereo_worker.py - Add anaglyph + left-eye extraction outputs
orchestrator.py  - Add delivery step, webhook callback, error recovery
video_service.py - Add HLS packaging after video generation
generate_showcase.py - Accept 3D creation prompt from Kimi

AUTOMATION FLOW:
User uploads images in Filament
-> Laravel creates PipelineJob -> Redis queue
-> orchestrator.py picks up job
-> [1] analyzer.py: image analysis + 3D creation prompt
-> [2] refiner.py: enhance key images (from prompt)
-> [3] generate_showcase.py: compile video (using prompt params)
-> [4] stereo_worker.py or splat_worker.py (based on prompt)
-> [5] resolver.py: color grade
-> [6] hls_packager.py: create HLS adaptive stream
-> [7] delivery.py: upload to S3, update DB
-> Laravel job complete -> notification sent

WHAT THE 3D CREATION PROMPT CONTROLS:
- animation_style -> how generate_showcase.py animates images
- transition_style -> how clips transition
- depth_strength -> stereo_worker.py depth parameters
- pipeline (sbs/splat/hybrid) -> which worker runs
- keyframe_indices -> which images refiner.py enhances
- clip_duration_per_image -> timing of each image in video
```

9.2 Pipeline B: Curved Screen Pixel Mapping Pipeline

PURPOSE: Maps the compiled video (from Pipeline A) onto physical curved or L-shaped LED screens so the image appears correctly 3D from the viewer sweet spot. Uses the 3D creation prompt to configure the anamorphic rendering.

```
FILES TO CREATE:
database/migrations/..._create_screen_geometries_table.php
app/Models/ScreenGeometry.php
app/Filament/Resources/ScreenGeometryResource.php
pipeline/uv_unwrapper.py
pipeline/shaders/distortion_correction.glsl
livestream/screen_{screen_id}.toe (TouchDesigner project)
app/Services/MediaServerService.php

AUTOMATION:
Admin sets screen geometry in Filament
-> UV unwrapper pre-computes UV map
-> TouchDesigner project auto-generated
-> Media server playlist updated
-> Content plays on schedule
```

-> Agentic Loop monitors playback quality

3D CREATION PROMPT CONTROLS:

- screen_placement.screen_type -> UV unwrapping algorithm
- screen_placement.sweet_spot_angle -> camera position
- screen_placement.viewing_distance_m -> perspective
- billboard_3d.inverted_box -> enable/disable box illusion
- billboard_3d.virtual_room_depth -> depth of virtual room
- billboard_3d.border_break_objects -> which objects leap out
- billboard_3d.forced_perspective -> enable/disable

9.3 Pipeline C: AI Video Generation Pipeline

PURPOSE: Generates additional video content using AI services - Veo for cinematic promos, Higgsfield for historical figure animation, Nano Banana 2 for image enhancement. Triggered by the 3D creation prompt from Kimi K3.

FILES EXISTING:

- pipeline/analyzer.py - Kimi K3 (prompt generation)
- pipeline/refiner.py - Nano Banana 2 (image enhancement)
- pipeline/flow_adapter.py - Google Veo 2.0 (promo videos)

FILES TO CREATE:

- pipeline/higgsfield_adapter.py - Higgsfield AI API
- pipeline/ai_content_router.py - routes to correct AI service
- pipeline/content_templates.json - prompt templates

AUTOMATION:

- Kimi generates 3D creation prompt
- > ai_content_router.py decides: Veo or Higgsfield?
- > flow_adapter.py or higgsfield_adapter.py runs
- > refiner.py enhances output
- > delivery.py uploads to CDN
- > Laravel: content ready

3D CREATION PROMPT CONTROLS:

- description -> used as text prompt for Veo/Higgsfield
- scene_type -> decides which AI service to use
- keyframe_indices -> starting images for video generation

9.4 Pipeline D: Mobile Delivery Pipeline

PURPOSE: Delivers the compiled video and 3D scenes to phone browsers in the optimal format. HLS adaptive streaming for video, compressed splatting for interactive 3D, PWA for offline.

FILES TO CREATE:

- pipeline/hls_packager.py - HLS adaptive streaming
- public/manifest.json - PWA manifest
- public/sw.js - Service worker
- public/3d/mobile_3d_viewer.js - Shared mobile controls module
- public/3d/player.html - Mobile video player with hls.js

FILES TO MODIFY:

- public/3d/kenya_3d_map.html - Add gyroscope + joystick
- public/3d/booth_viewer.html - Add gyroscope + joystick
- public/3d/sector_map.html - Add gyroscope + joystick

AUTOMATION:

- Pipeline A completes (video generated)
- > hls_packager.py runs automatically
- > HLS segments uploaded to S3/CDN
- > Laravel updates video URL to .m3u8 playlist
- > PWA service worker caches for offline
- > Phone users see adaptive quality

9.5 Pipeline E: Live 3D Streaming Pipeline

PURPOSE: The only pipeline that uses live video instead of images. Captures live events from dual stereo cameras and streams in real-time 3D to phones and VR headsets.

FILES TO CREATE:

- livestream/obs_scene_sbs.json - OBS scene template
- deploy/streaming-server/docker-compose.yml - nginx-rtmp
- deploy/streaming-server/nginx.conf - nginx config
- app/Models/Livestream.php - Eloquent model
- database/migrations/..._create_livestreams_table.php
- app/Filament/Resources/LivestreamResource.php - Admin UI
- app/Http/Controllers/Web/LiveController.php - Viewing page

```
app/Services/LivestreamService.php      - Cloudflare API
public/3d/live_player.html              - Web VR/3D/2D player
resources/js/live-chat.js               - WebSocket chat
```

AUTOMATION:

```
Organizer clicks "Go Live" in Filament
-> Laravel creates Livestream (status: starting)
-> Cloudflare Stream API: create live input -> RTMP URL
-> OBS auto-starts with scene config
-> OBS pushes RTMP to Cloudflare
-> Laravel detects stream live (status: live)
-> Exhibition page shows "Watch Live" button
-> Viewers connect -> HLS plays in browser
-> WebSocket tracks viewer count
-> When event ends -> recording saved for replay
```

10. COMPLETE SOFTWARE STACK - PAID VS FREE

10.1 Free / Open-Source Software

Software	Used In	License
Three.js	All 3D web experiences	MIT
3D Gaussian Splatting	3D scene reconstruction	MIT/BSD
Depth Anything V2	Depth map generation	Apache 2.0
COLMAP	Camera pose estimation	BSD
TRELLIS.2 (Microsoft)	Image to 3D model	MIT
FFmpeg	Video processing, HLS, anaglyph	LGPL/GPL
Blender	3D modeling, animation	GPL
Unreal Engine 5	Real-time rendering	Free (5% royalty >\$1M)
Laravel 13	Backend framework	MIT
Filament	Admin panel	MIT
Python + OpenCV	Pipeline workers	BSD
OBS Studio	Live capture + compositing	GPL
nginx + nginx-rtmp	Streaming server	BSD
Cloudflare Free Tier	CDN, DDoS, SSL	Free tier
hls.js	HLS video player	Apache 2.0
Video.js	Mobile video player	Apache 2.0
nipplejs	Virtual joystick	MIT
gsplat.js	Web splat viewer	MIT
SQLite / MySQL	Database	Public/GPL
Pillow (PIL)	Image processing	Historical

10.2 Paid Software

Software	Cost	Type	Use Case
OpenRouter (Kimi K3)	\$50-200/mo	AI API	Image analysis + 3D creation prompt
Google AI Pro (Imagen + Veo)	\$20/mo	AI API	Nano Banana 2 enhance + Veo promos
Higgsfield AI	\$30-100/mo	AI API	Historical figure animation, video gen
DaVinci Resolve Studio	\$295 once	Software	Color grade, edit, Fusion compositing
Vast.ai / RunPod GPU	\$500-2,000/mo	Cloud GPU	DAv2, 3DGS, TRELLIS training
TouchDesigner Commercial	\$50/mo	Software	Pixel mapping, UV unwrapping
PIXERA	\$40/mo	Software	Media server, Genlock sync
Disguise (alt to PIXERA)	\$3,000-15,000	Software	Mission-critical media server
SideFX Houdini Indie	\$50/mo	Software	Procedural assembly animation
After Effects	\$55/mo	Software	Motion graphics, compositing
Unity Pro	\$2,040/yr	Software	Native VR app development
Cloudflare Stream	\$10/mo	API	Live streaming CDN + transcoding
Novastar LED Processor	\$500-5,000	Hardware	LED panel controller
Blackmagic DeckLink Duo 2	\$695	Hardware	Dual SDI capture card
Stereo Camera Pair	\$1,500-6,000	Hardware	Genlocked cameras for live 3D
Meta Quest 3	\$500-700	Hardware	VR headset per booth
Laravel Forge	\$15/mo	DevOps	Server management
DigitalOcean/S3	\$50/mo	Cloud	Storage + CDN
Sentry	\$0-50/mo	Monitoring	Error tracking

11. COST SUMMARY

11.1 Monthly Recurring Costs

Service	Monthly Cost	Required For
OpenRouter (Kimi K3)	\$50-200	Image analysis + 3D creation prompt
Google AI Pro	\$20	Nano Banana 2 enhance + Veo promos
Higgsfield AI	\$30-100	Historical figure animation, video gen
Vast.ai GPU (A100)	\$500-1,000	Depth estimation, 3DGS, TRELLIS
TouchDesigner	\$50	Curved screen pixel mapping
PIXERA	\$40	Media server for airport screens
Houdini Indie	\$50	Procedural assembly animation
After Effects	\$55	Motion graphics, maps, text overlays
Cloudflare Stream	\$10	Live streaming CDN
Laravel Forge	\$15	Server management
DO/S3 Storage + CDN	\$50	Asset delivery
Sentry Monitoring	\$0-50	Error tracking

BASE AI: \$70-320/mo (Kimi + Google + Higgsfield)

+GPU: \$500-1,000/mo (Vast.ai splatting)

+DISPLAY: \$140/mo (TouchDesigner + PIXERA + Houdini + AE)

+LIVE STREAM: \$10/mo (Cloudflare Stream)

TOTAL: \$720-1,470/mo

11.2 One-Time Costs

Item	Cost	Notes
DaVinci Resolve Studio	\$295	One-time license
Meta Quest 3 (per booth)	\$500-700	Quantity depends on booth count
Novastar LED Processor	\$500-5,000	Per screen installation
Unity Pro	\$2,040/yr	Annual, only if native VR needed
LED Screens (per sq meter)	\$1,000-3,000	Hardware cost per screen
VR Booth enclosure	\$2,000-5,000	Physical booth structure
Stereo Camera Pair	\$1,500-6,000	For live 3D streaming
Blackmagic DeckLink Duo 2	\$695	Capture card for live 3D
RTX 4090 Streaming PC	\$3,000-5,000	Live streaming workstation

11.3 Total Project Cost Estimate (18 Months)

Category	Monthly	Total
Software subscriptions	\$720-1,470/mo	\$12,960-26,460
One-time software licenses	N/A	\$295-4,295
Hardware (LED, VR, cameras)	N/A	\$53,000-166,000
Development labor (3-4 devs)	\$15,000-25,000/mo	\$270,000-450,000 (6mo)
Cloud infrastructure	\$50-100/mo	\$900-1,800
Live streaming hardware	N/A	\$5,195-11,695

TOTAL SOFTWARE & INFRASTRUCTURE: \$64,000-182,000 over 18 months

Plus hardware: \$53,000-166,000

12. PRIORITY BUILD ORDER - EASIEST TO MOST COMPLEX

The order in which to build the pipelines, ranked from easiest/cheapest to most complex/expensive. Each step builds on the previous one.

12.1 Priority 1: Mobile Delivery Pipeline (Weeks 1-2)

COST: \$0/mo new (all free software)
 EXISTING: 3 Three.js HTML files, OrbitControls, touch handlers
 NEW FILES: hls_packager.py, manifest.json, sw.js, mobile_3d_viewer.js, player.html
 MODIFIED: kenya_3d_map.html, booth_viewer.html, sector_map.html

WHAT YOU GET:

- Gyroscope tilt-to-look on phone (DeviceOrientation API)
- Virtual joystick for movement (nipplejs)
- HLS adaptive streaming (video quality adjusts to connection)
- PWA: install on phone home screen, offline caching
- 3 mode toggle: 2D / 3D Anaglyph / VR Split-screen

WHY FIRST: Zero new paid services. Everything is free CDN libraries and browser APIs. Builds directly on the 3 Three.js files that already work. Immediate value - phone users can already access the 3D experiences.

12.2 Priority 2: AI Video Generation Pipeline (Weeks 3-5)

COST: \$70-320/mo (Kimi + Google + Higgsfield API keys)
 EXISTING: analyzer.py (124 lines), refiner.py (84 lines), flow_adapter.py (110 lines)
 NEW FILES: higgsfield_adapter.py, ai_content_router.py, content_templates.json

WHAT YOU GET:

- Kimi K3 generates 3D creation prompt from uploaded images
- Nano Banana 2 enhances images to 4K
- Google Veo generates promotional videos
- Higgsfield AI animates historical figures from photos
- AI content router automatically chooses best AI service

WHY SECOND: 3 of 5 components are already coded and working. Just need API keys. Only the Higgsfield adapter and content router are new files. The main cost is API subscriptions.

12.3 Priority 3: 3D Content Pipeline Core (Weeks 5-8)

COST: \$500-1,000/mo (Vast.ai GPU)
 EXISTING: depth_worker.py (131), stereo_worker.py (158), splat_worker.py (146 scaffolded), orchestrator.py (201), resolver.py (101), generate_showcase.py (252)
 NEW FILES: hls_packager.py, delivery.py
 MODIFIED: splat_worker.py (train_3dgs + compress), stereo_worker.py (anaglyph), orchestrator.py (delivery + webhook), video_service.py (HLS)

WHAT YOU GET:

- Complete pipeline: images -> 3D creation prompt -> video -> SBS 3D -> splat
- GPU-accelerated depth estimation (DAV2)
- Full 3D Gaussian Splatting training (not placeholder)
- Splat compression for web delivery
- SBS 3D video + anaglyph + HLS streaming
- DaVinci Resolve color grading (or FFmpeg fallback)

WHY THIRD: 6 of 8 components already coded. The two gaps are:
 (1) actual 3DGS training code (currently placeholder), and
 (2) splat compression for web. Main blocker is GPU compute cost.

12.4 Priority 4: Live 3D Streaming Pipeline (Weeks 8-12)

COST: \$10/mo (Cloudflare Stream) + \$5,195-11,695 one-time hardware
 EXISTING: Nothing (only database has_livestream field in SubscriptionPlan)
 NEW FILES: 8 files (OBS template, Docker, nginx, Laravel Livestream model, migration, Filament resource, LiveController, LivestreamService, live_player.html, live-chat.js)

WHAT YOU GET:

- Real-time stereo 3D streaming from live events
- Phone viewers: 2D or anaglyph 3D
- VR headset viewers: full SBS 3D
- On-demand replay after event ends

- Live viewer count via WebSocket

WHY FOURTH: Significant hardware investment (\$5,000-12,000) and 8 new components to build from scratch. But the software pieces are well-understood (OBS, nginx-rtmp, HLS, Cloudflare API).

12.5 Priority 5: Curved Screen Pixel Mapping Pipeline (Weeks 12-18)

COST: \$90/mo (TouchDesigner \$50 + PIXERA \$40) + \$500-5,000 Novastar HW
 EXISTING: Nothing
 NEW FILES: ScreenGeometry migration + model + resource, uv_unwrapper.py, distortion_correction.glsl, .toe files, MediaServerService.php

WHAT YOU GET:

- 3D billboard effect on curved/L-shaped LED screens
- Inverted box illusion (Shinjuku cat style)
- UV unwrapping for any screen shape
- Distortion correction for screen curvature
- Seam blending between LED panels
- Genlock sync across multiple screens
- Remote CMS scheduling and monitoring

WHY FIFTH: Most complex software development (UV unwrapping math, custom GLSL shading, TouchDesigner node graph) AND hardware-dependent (Novastar LED processor, physical screen installation). Requires specialized skills plus physical hardware procurement and installation.

12.6 Summary Build Timeline

WEEK	PIPELINE	COST (monthly)
1-2	Mobile Delivery	\$0
3-5	AI Video Gen	\$70-320
5-8	3D Content Core	\$500-1,000
8-12	Live 3D Streaming	\$10 + \$5-12K HW
12-18	Curved Screen	\$90 + \$500-5K HW
TOTAL ONGOING		\$670-1,410/mo

This order means:

- Week 2: Phone users get gyroscope-powered 3D exploration (FREE)
- Week 5: AI generates 3D creation prompts from uploaded images
- Week 8: Full pipeline: images in -> 3D video out (GPU-powered)
- Week 12: Live events stream in real-time 3D
- Week 18: Curved LED billboards display 3D content with inverted box illusion

A. APPENDIX A: EXISTING PIPELINE FILE REFERENCE

Complete reference of all existing pipeline files in kenya-3d-platform/:

```

pipeline/
+- analyzer.py           124 lines  Kimi K3 image analysis + 3D prompt gen
+- depth_worker.py       131 lines  Depth Anything V2 depth estimation
+- stereo_worker.py      158 lines  SBS 3D video generation
+- splat_worker.py       146 lines  3D Gaussian Splatting (SCAFFOLDED)
+- refiner.py            84 lines   Nano Banana 2 / Imagen enhancement
+- resolver.py           101 lines  DaVinci Resolve API + FFmpeg grading
+- flow_adapter.py       110 lines  Google Veo 2.0 promo generation
+- orchestrator.py       201 lines  Master pipeline controller
+- orchestrator_v2.py    375 lines  47-county 3D map pipeline
+- video_service.py      288 lines  Dynamic image compilation + screen presets
+- scene_composer.py     --        Three.js county map HTML generator
+- sector_map_composer.py --        Sector explorer HTML generator
+- terrain_gen.py        --        2D image -> 3D terrain mesh
+- county_data.py        344 lines  All 47 Kenyan counties metadata
+- image_analyzer.py     --        Pillow image analysis
+- media_reader.py       --        Image/video reading utilities
+- audio_extractor.py    --        Audio extraction utility
+- webgl_viewer/        Generated 3D HTML viewer output
  +- kenya_3d_map.html   453 lines  47-county interactive 3D map
  +- sector_map.html     277 lines  Mombasa/Kilifi sector explorer
  +- booth_viewer.html   361 lines  Exhibition hall 3D tour
  +- player.js           WebGL player
  +- booth-overlay.js    Booth interaction overlay

scripts/
+- generate_showcase.py  252 lines  Image -> video compilation (Ken Burns)
+- register_images.py    --        Image registry management

agentic_loop/
+- agentic_loop.py       Main O-D-A-L loop runner
+- observer.py           Observes SEO, pipeline, weather, users
+- decider.py            Kimi K3 decision making
+- actor.py              Executes actions (write JSON for Laravel)
+- learner.py            RL-based learning from outcomes
+- workflow_engine.py    n8n-compatible workflow executor

seo-engine/
+- Detector.php          219 lines  Geo-weather-trend detection
+- DestinationMatcher.php 173 lines  Cross-reference user + Kenya data

cloudflare-worker/
+- worker.js             20 lines   R2 media proxy + Pages redirect
+- wrangler.jsonc        Wrangler config

n8n-workflows/
+- county-content-pipeline.json  Auto content ingestion
+- international-trade-promotion.json  Trade promotion
+- sbs-pipeline.json           SBS video pipeline

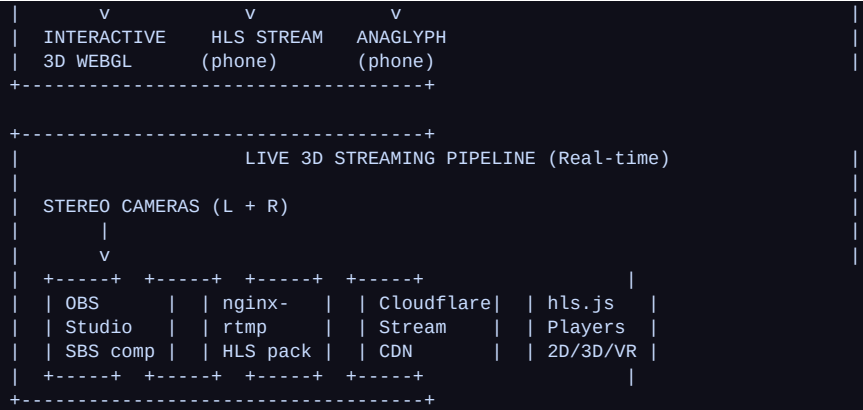
deploy/
+- deploy_vast.py        Vast.ai auto-deploy
+- docker-compose.yml    Docker services
+- Dockerfile            GPU worker container

```

B. APPENDIX B: INTEGRATION ARCHITECTURE

How all 7 use cases connect through the platform. The core data flow is: USER UPLOADS IMAGES -> 3D CREATION PROMPT -> COMPILED VIDEO -> 3D CONVERSION -> DELIVERY.





 END-TO-END FLOW: User uploads images -> 3D video on billboard

1. User uploads 20 photos of Mt. Kenya via Filament admin panel
2. Laravel registers images in image_registry.db
3. Laravel creates PipelineJob -> Redis queue
4. orchestrator.py picks up job:
 - a. analyzer.py (Kimi K3): analyzes each image, generates 3D creation prompt


```
{
    "scene_type": "walkthrough",
    "depth_strength": 2.0,
    "pipeline": "hybrid",
    "animation_style": "zoom_in_to_peak",
    "inverted_box": true,
    "border_break_objects": ["mountain_peak"]
  }
```
 - b. refiner.py (Nano Banana 2): enhances 5 key images to 4K
 - c. generate_showcase.py: compiles 20 images into 120s video (uses prompt: animation_style, transition_style, timing)
 - d. stereo_worker.py: converts video to SBS 3D (uses prompt: depth_strength=2.0, convergence=0.4)
 - e. splat_worker.py: builds interactive 3D scene from images (uses prompt: pipeline="hybrid")
 - f. hls_packager.py: creates adaptive HLS stream for phones
 - g. delivery.py: uploads all outputs to S3, updates DB
5. Laravel marks job complete
6. Video is now available:
 - VR booth: downloads SBS video, plays in WebXR
 - Phone: HLS adaptive stream (2D) or anaglyph (3D) or WebGL (interactive)
 - Curved LED billboard: TouchDesigner maps SBS video to screen geometry with inverted box illusion, forced perspective from sweet spot
7. [SEPARATE] Human voiceover artist records narration
 Audio mixed into final video as a separate post-production step

End of Document - KICC 3D Billboard Platform Architecture v1.0

7 Use Cases | Core: Images In -> 3D Creation Prompt -> Video Out -> 3D Conversion -> Delivery